

Plan de Game Day: ingeniería del caos sobre un pipeline de observabilidad OpenTelemetry

Hipótesis de fallo, diseño de experimentos, métricas de verificación y resultados de la ejecución · Ilich · Maestría — Observabilidad y SRE (MASOBAP2026), Unidad 2 · Actividad individual · 30 de agosto de 2026

1. Arquitectura objetivo, dependencias y estado estable

El sistema objetivo es el laboratorio construido en la actividad anterior (`otel-e2e-lab`): un pipeline de observabilidad de extremo a extremo sobre dos microservicios FastAPI instrumentados con OpenTelemetry, que emite los tres pilares y los correlaciona por `trace_id` (W3C TraceContext). La cadena de petición es cliente → **service-a** (órdenes) → **service-b** (inventario) → **PostgreSQL**, y la telemetría fluye por OTel Collector → Jaeger (trazas) / Prometheus (métricas) / Loki (logs).

Componente	Rol en la cadena de petición	Dependencias	Modo de fallo dominante
service-a (órdenes) FastAPI, 2 workers	Punto de entrada. Consulta stock y precio, calcula total con IVA y descuento (<code>span orders.calculate_total</code>).	service-b vía HTTP (<code>httpx.Client</code> , <code>timeout 10 s</code> , sin reintento)	Endpoints síncronos: la espera de la dependencia bloquea hilos del <code>threadpool</code> ($2 \times 40 = 80$ ranuras).
service-b (inventario)	Verifica existencias y precio (<code>span inventory.check_stock</code>).	PostgreSQL 16 vía SQLAlchemy (<code>pool_size=20, max_overflow=40</code>)	Sin <code>connect_timeout</code> ni <code>statement_timeout</code> : una partición silenciosa bloquea la conexión.
PostgreSQL 16	Almacén de inventario (5 SKU sembrados).	—	Único punto de estado; sin réplica de lectura.
OTel Collector 0.116.1	<code>memory_limiter</code> → <code>resource</code> → <code>batch</code> ; exporta a Jaeger, Prometheus y Loki.	Backends de telemetría	Camino de observación: su caída deja al sistema ciego, no caído.

Estado estable (línea base medida, 120 s / 4 801 peticiones a 40 rps): `p50 = 16,0 ms`; `p95 = 21,9 ms`; `p99 = 36,3 ms`; disponibilidad 100 %; peticiones en vuelo ≈ 1. Sobre esta base se derivan los SLO que los experimentos ponen a prueba: **latencia `p95` ≤ 250 ms** (holgura de 11× sobre la línea base) y **disponibilidad ≥ 99,5 %** mensual, equivalente a un error budget de 216 minutos al mes.

2. Hipótesis de fallo

Las tres hipótesis se formulan sobre comportamiento observable y medible, y cada una nombra explícitamente el estado estable de partida, la condición de fallo y el resultado esperado. Son afirmaciones falsables: el valor del Game Day está en la predicción previa, no en provocar caídas.

#	Hipótesis
H1	En estado estable — <code>p95</code> de 21,9 ms, 100 % de disponibilidad y ≈1 petición en vuelo a 40 rps—, cuando se inyecten 400 ms de latencia adicional en el enlace <code>service-a</code> → <code>service-b</code> , esperamos que el sistema conserve la disponibilidad por encima del 99,5 %, que el <code>p95</code> extremo a extremo se degrade de forma proporcional y acotada por debajo de 1 s, y que no haya saturación del pool de hilos ni pérdida de throughput.
H2	En estado estable —mismas condiciones—, cuando se agote la CPU asignada a <code>service-b</code> (cuota de 1 vCPU disputada por cuatro hilos de <code>stress-ng</code>), esperamos que la degradación quede contenida en <code>service-b</code> , que el <code>p95</code> extremo a extremo permanezca por debajo del SLO de 250 ms y que throughput y disponibilidad no varíen.
H3	En estado estable —mismas condiciones—, cuando se produzca una partición de red silenciosa (descarte de paquetes, sin RST) entre <code>service-b</code> y PostgreSQL, esperamos que <code>service-b</code> detecte la indisponibilidad y devuelva 503 en menos de 2 s, que <code>service-a</code> lo traduzca en un 502 rápido, y que el fallo sea rápido y acotado, sin agotar pools ni propagar saturación aguas arriba.

3. Diseño de los experimentos de caos

Los tres experimentos se ejecutan en un entorno reconstruido a partir del repositorio del laboratorio, **nunca en producción**, y cada uno lleva dos mecanismos de reversión independientes: un temporizador duro que retira la falla al cerrar la ventana pactada y un watchdog que sondea un canario cada 2 s y aborta anticipadamente si se cumple la condición de parada. La escalada del radio de impacto es deliberada: E1 comienza con una degradación moderada y solo escala si el sistema la absorbe. Las ventanas de 60–120 s no son arbitrarias: son lo bastante largas para cubrir al menos doce intervalos de scrape de Prometheus (5 s) y varios ciclos de exportación OTLP (10 s) —de modo que el efecto quede registrado en las series temporales y no solo en el cliente— y lo bastante cortas para que, si la hipótesis se refuta, el consumo de error budget se mantenga por debajo del 1 % del presupuesto mensual.

#	Tipo de fallo	Inyector	Radio de impacto	Duración	Rollback automático
E1	Latencia de red	Toxiproxy 2.11, toxina <code>latency</code> (equivalente a <code>tc netem</code>)	Solo el socket TCP <code>service-a</code> → <code>service-b</code> . Postgres, Collector, Prometheus, Jaeger y el tráfico del cliente quedan fuera.	120 s estable / 90 s a 400 ms / 90 s a 2 500 ms / 120 s de observación	Retiro de la toxina por temporizador; watchdog que aborta ante 5 canarios seguidos con error o > 8 s.
E2	Resource exhaustion (CPU)	<code>stress-ng</code> 0.17.06, 4 hilos <code>matrixprod</code>	cgroup con cuota de 1 vCPU que contiene únicamente los procesos de <code>service-b</code> y los de <code>stress-ng</code> .	60 s estable / 120 s de inyección / 90 s de observación	SIGKILL al grupo de procesos por temporizador; misma condición de parada del watchdog.
E3	Network partition	<code>iptables</code> ... -j DROP dentro del network namespace	Solo el tráfico saliente de <code>service-b</code> hacia el puerto 5432. PostgreSQL sigue vivo y atendiendo.	60 s estable / 90 s de partición / 90 s de observación	Borrado de la regla por temporizador; watchdog con umbral ampliado (se espera error) que vigila que la degradación no exceda la ventana.

4. Métricas de observabilidad para verificar cada hipótesis

Cada hipótesis se verifica con un **SLI primario** que mide el impacto, un **SLI de control** que discrimina dónde está el problema, y señales de **logs** y **trazas** que confirman el mecanismo. Todas las métricas provienen de la instrumentación real del laboratorio, exportadas por el Collector y consultadas en Prometheus; el nombre base de la serie es `http_server_duration_milliseconds`.

Hip.	SLI primario y de control	Consulta PromQL	Logs y trazas que confirman el comportamiento
H1	Latencia p95 extremo a extremo (impacto) y peticiones en vuelo (saturación)	<code>histogram_quantile(0.95, sum by (le) (rate(..._bucket{job="service-a"}[5m])) . sum(http_server_active_requests{job="service-a"}))</code>	Traza en Jaeger de GET <code>/api/orders/{item_id}</code> : comparar la duración del span cliente HTTP con la del span servidor de service-b. Si el segundo es corto, el retardo está en el enlace o en la cola, no en la dependencia.
H2	Latencia p95 de service-b (impacto) y throughput (control)	<code>histogram_quantile(0.95, sum by (le) (rate(..._bucket{job="service-b"}[5m])) . sum(rate(..._count{job="service-a"}[1m]))</code>	Histograma propio <code>inventory_check_duration_ms</code> y span <code>inventory.check_stock</code> : aíslan el coste de CPU de la lógica de negocio del de la consulta SQL. <code>cpu.stat</code> del cgroup como evidencia de throttling.
H3	Disponibilidad (impacto), tiempo hasta el fallo y saturación de ambos servicios	<code>1 - sum(rate(..._count{job="service-a", http_status_code=~"5.."}[5m])) / sum(rate(..._count{job="service-a"}[5m]))</code>	Log JSON de service-b «Base de datos inalcanzable» con su <code>trace_id</code> ; span <code>SELECT inventory</code> y span <code>connect</code> de SQLAlchemy. La ausencia de ese log es en sí misma una señal de diagnóstico.

5. Ejecución y evidencias

5.1 Procedimiento y herramientas

El laboratorio se reconstruyó con el código fuente original y las mismas versiones del stack (OTel Collector 0.116.1, Prometheus 3.1.0, Jaeger 1.62.0, PostgreSQL 16), ejecutadas de forma nativa en lugar de docker-compose. service-b se aisló en un **network namespace** unido al host por un par veth, de modo que el radio de impacto fuera *verificable* y no meramente declarado. La carga se generó en **circuito abierto a 40 rps constantes** —tasa de llegada independiente de la latencia de respuesta—, condición sin la cual una degradación se auto-oculta al reducir la carga ofrecida. Se ejecutaron los tres experimentos; 16 800 peticiones medidas en el cliente.

```
# E1 – latencia (Toxiproxy; en el laboratorio local, tc netem)
curl -XPOST localhost:8474/proxies/inventory/toxics -d '{"name":"lat","type":"latency",
  "stream":"downstream","attributes":{"latency":400,"jitter":100}}'
curl -XDELETE localhost:8474/proxies/inventory/toxics/lat # rollback
tc qdisc add dev eth0 root netem delay 400ms 100ms distribution normal # equivalente nativo

# E2 – agotamiento de CPU acotado por cgroup
echo 100000 > /sys/fs/cgroup/cpu/svcb/cpu.cfs_quota_us # cuota = 1 vCPU
stress-ng --cpu 4 --cpu-method matrixprod --timeout 130s # dentro del cgroup

# E3 – partición de red silenciosa hacia PostgreSQL
ip netns exec svcb iptables -A OUTPUT -p tcp --dport 5432 -j DROP
ip netns exec svcb iptables -D OUTPUT -p tcp --dport 5432 -j DROP # rollback
```

NOTA SOBRE LA HERRAMIENTA DE LATENCIA

El kernel del entorno de ejecución (Firecracker 6.18, sin módulos cargables) no expone `sch_netem`, por lo que `tc netem` se substituyó por Toxiproxy, que aplica el mismo modelo delay + jitter sobre el enlace. E2 y E3 sí usan las herramientas nativas del enunciado. La reproducción de §5.3 sí ejecuta `netem`, a través de Chaos Mesh.

5.2 Evidencia del Experimento 1: métricas, trazas y logs



Figura 1. La latencia inyectada (bandas sombreadas) atraviesa el sistema sin amortiguación: el p50 pasa de 16,0 ms a 417,4 ms con 400 ms inyectados —propagación 1:1— y a 4 153,5 ms con 2 500 ms inyectados, 1,66× el retardo. Escala logarítmica; 16 800 peticiones medidas en el cliente.

Fase	Peticiones	Throughput	Disponib.	p50	p95	En vuelo
Estado estable	4 801	40,0 rps	100 %	16,0 ms	21,9 ms	1
Fase 1 — 400 ms	3 601	40,0 rps	100 %	417,4 ms	508,5 ms	19
Fase 2 — 2 500 ms	2 660	29,5 rps	100 %	4 153,5 ms	6 008,8 ms	121
Tras el rollback	5 738	47,8 rps	99,95 %	17,4 ms	1 396,8 ms	4

La **traza** cierra el diagnóstico. Para una petición de 6 136,7 ms durante la fase 2, el span cliente HTTP hacia service-b vale **2 962,4 ms**, pero el span servidor de service-b vale **7,8 ms** y el `SELECT inventory`, **0,4 ms**: la dependencia está sana y el tablero centrado en ella marcaba 22 ms durante todo el incidente. Los **3 174 ms restantes no pertenecen a ningún span**: son espera en la cola del pool de hilos. El **log** estructurado permite pivotar de la métrica a la traza y de la traza al log sin cambiar de herramienta:

```
{"timestamp": "2026-08-29T18:05:53.564+00:00", "severity": "INFO", "service": "service-b",
  "message": "Stock verificado para SKU-004", "trace_id": "10ddcda0faae6c8720c10a83a362ec44",
  "span_id": "92bfl147658255138", "item_id": 4}
```

5.3 Reproducción sobre Kubernetes con Chaos Mesh y verificación de la remediación

Para comprobar que los hallazgos no dependen del entorno, el mismo diseño se reprodujo sobre un **cluster kubeadm de tres nodos** con Chaos Mesh como inyector: un `NetworkChaos` de 200 ms ±10 ms (netem) equivalente a E1, y un `HTTPChaos` de error rate del 10 % que amplía el catálogo con el tipo `error`. Se añadió un **blackbox exporter** que sondea cada 5 s desde fuera del proceso instrumentado, precisamente para cubrir el punto ciego que reveló E1.

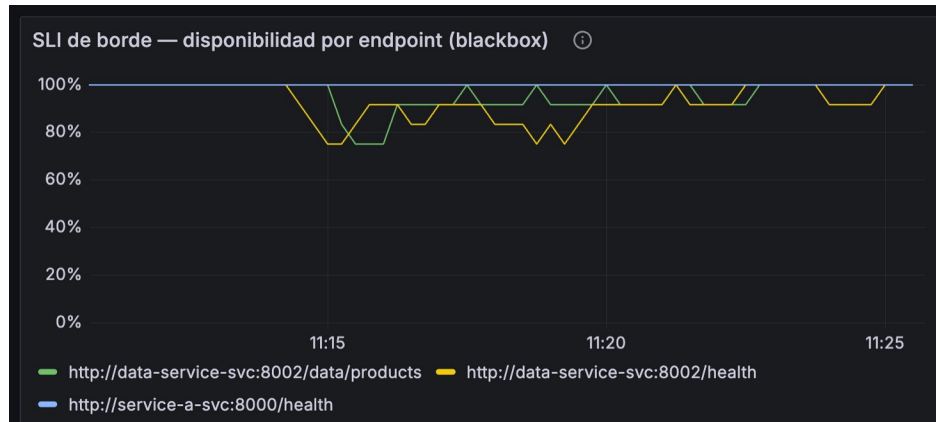


Figura 2. Tablero de Grafana durante la inyección de error rate. La sonda externa detecta la caída de disponibilidad en tiempo real (verde y amarillo), mientras service-a, fuera del blast radius, permanece en 100 % (azul). Es la única fuente del tablero que registró la falla: el APM interno informó 200 OK durante toda la ventana.

REMEDIACIÓN VERIFICADA EXPERIMENTALMENTE

La primera corrida dejó el health check *dentro* del blast radius: el chaos abortó también GET /health, el kubelet mató el contenedor cada 77 s y el rollback automático nunca se ejecutó (último error en t+466,8 s frente a un TTL de 300 s; 6 reinicios; 19 eventos Recover / Failed). Una corrida de control con el health check *fuera* del blast radius —un único campo del manifiesto— cerró el experimento 2 s antes del fin nominal, con 0 reinicios y 0 errores después del TTL, sin perder efectividad en la inyección (11,8 % de error frente al 10 % nominal).

6. Resultados esperados frente a resultados obtenidos

Cada experimento rompe un SLI distinto

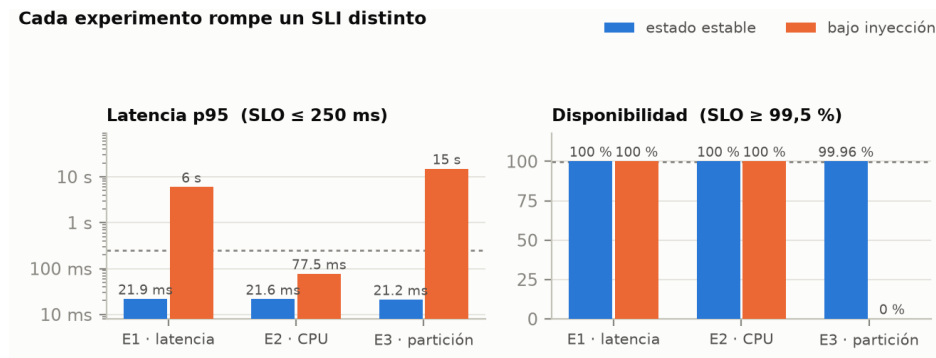


Figura 3. Los tres experimentos rompen SLIs distintos: E1 y E3 rompen la latencia; solo E3 rompe la disponibilidad; E2 no rompe ninguno. Un régimen de alertas basado únicamente en 5xx habría sido ciego a dos de los tres modos de fallo.

Hip.	Resultado esperado	Resultado obtenido	Veredicto
H1	Disponibilidad > 99,5 %, p95 < 1 s, sin saturación ni pérdida de throughput.	Disponibilidad 100 % ✓. Con 400 ms: p95 = 508,5 ms (SLO superado 2x) ✗. Con 2 500 ms: p95 = 6 008,8 ms, amplificación de 1,66x sobre el retardo inyectado, throughput caído de 40,0 a 29,5 rps (-26 %) y 121 peticiones en vuelo frente a 80 ranuras ✗.	REFUTADA
H2	Degradación contenida en service-b, p95 < 250 ms, throughput y disponibilidad intactos.	p95 de 21,6 a 77,5 ms (3,6x, dentro del SLO) ✓; p95 de service-b de 22,1 a 56,2 ms ✓; throughput 40,0 rps constante ✓; disponibilidad 100 % ✓; recuperación inmediata al retirar la carga ✓. El cgroup registró 1 189 de 3 158 periodos con throttling (87,2 s), confirmando que el fallo se inyectó de verdad.	CONFIRMADA
H3	Fallo rápido: 503 desde service-b en menos de 2 s, 502 en service-a, sin agotar pools.	Disponibilidad 0 % durante los 90 s ✗. Ningún fallo rápido: p50 = 15 011,9 ms, el tope del cliente. 342 respuestas 503 y 560 timeouts de lectura. Saturación masiva: 179 peticiones en vuelo en service-a y 717 en service-b ✗. A los 90 s del rollback el sistema aún no había vuelto al estado estable ✗.	REFUTADA

7. Debilidades sistémicas reveladas y remediación propuesta

D1 — No existe un presupuesto de tiempo por salto. El cliente HTTP de service-a lleva un único timeout de 10 s, sin reintento ni cortocircuito, cuando el p99 real de la dependencia es de 36 ms. Cualquier lentitud aguas abajo se propaga íntegra al usuario y, superado el punto de saturación, se amplifica 1,66x por encolamiento.

D2 — El SLI de disponibilidad es ciego al modo de fallo más probable. Durante los 180 s de E1 no se registró un solo 5xx: la disponibilidad marcó 100 % mientras el SLO de latencia se incumplía por un factor de 24, y el tablero de la dependencia mostraba 22 ms. Solo la combinación de latencia extremo a extremo + peticiones en vuelo + traza reveló el incidente.

D3 — El manejo de error de base de datos es código muerto ante una partición silenciosa. service-b contiene un except `OperationalError` → 503 deliberado, pero en E3 no se escribió una sola línea de «Base de datos inalcanzable»: sin `connect_timeout` ni `statement_timeout` el driver nunca lanza la excepción. El código defensivo existe, pero es inalcanzable justo en el escenario para el que se escribió.

D4 — No hay aislamiento por dependencia ni control de admisión. Una sola dependencia degradada consume el pool de hilos completo (E1: 121 en vuelo; E3: 717 en service-b) y la recuperación no es instantánea: el drenaje de la cola mantuvo el p95 en 1,4 s durante 20 s tras el rollback de E1. Los 90 s de indisponibilidad de E3 consumen el **0,69 %** del error budget mensual; ese mismo fallo sostenido una hora en producción consumiría el **27,8 %** en un solo incidente.

Pri.	Acción de remediación	Implementación concreta	Efecto esperado y verificación
1	Presupuesto de tiempo por salto	<code>httpx.Timeout(connect=0.25, read=0.8)</code> en service-a ($\approx 20\times$ el p99 medido) y <code>connect_args={"connect_timeout": 2, "options": "-c statement_timeout=2000"}</code> en el motor SQLAlchemy de service-b.	Cierra D1 y D3: el fallo se manifiesta en menos de 1 s en lugar de 6–15 s, y activa el manejador de <code>OperationalError</code> . Se verifica repitiendo E1 y E3.
2	Circuit breaker con degradación elegante	Cortacircuitos en el cliente de inventario (abrir tras 5 fallos o $p95 > 1$ s en 30 s; semiabierto a los 15 s) devolviendo precio de catálogo cacheado con <code>in_stock: "unknown"</code> .	Convierte una caída total en degradación parcial. Verificación: E3 debe cerrar con disponibilidad $> 99,5\%$ en lugar de 0 %.
3	Bulkhead y control de admisión	<code>httpx.Limits(max_connections=20)</code> más un semáforo por dependencia, y rechazo rápido (429) al superar el 80 % del pool de hilos.	Cierra D4: acota el encolamiento y elimina la amplificación de $1,66\times$.
4	SLI medido fuera del proceso y alertas por error budget	Blackbox exporter sondeando cada 5 s (ya desplegado en la reproducción de §5.3), SLI de saturación sobre <code>http_server_active_requests</code> y alertas multiventana (14,4× en 1 h, 6× en 6 h) sobre latencia y disponibilidad.	Cierra D2: los tres experimentos habrían disparado alerta; hoy, dos de tres pasan inadvertidos. Ya implementado y en uso.
5	Excluir el health check del blast radius	Acotar el path del experimento al tráfico de negocio, o usar una <code>livenessProbe</code> de tipo <code>exec/tcpSocket</code> que no atraviese el inyector.	Restaura el rollback automático. Verificada experimentalmente en la corrida de control de §5.3: 6 reinicios $\rightarrow 0$ y desfase de $+166,8$ s $\rightarrow -2,0$ s.
6	Institucionalizar el Game Day	Cadencia mensual con LitmusChaos o Chaos Mesh, runbook que arranca por el pivote <code>trace_id</code> y postmortem sin culpables.	Convierte los hallazgos en práctica cultural (anti-fragilidad); el indicador de éxito es la reducción sostenida del MTTR.

El resultado más valioso del ejercicio no es que dos hipótesis fueran refutadas, sino *por qué* lo fueron: el sistema estaba **bien instrumentado y mal protegido**. La observabilidad hizo visible en segundos un fallo que el régimen de alertas vigente no habría notificado nunca, y esa asimetría —ver mucho, reaccionar poco— es exactamente la debilidad sistémica que un Game Day existe para descubrir antes de que lo haga producción.

Referencias

Basiri, A., Behnam, N., de Rooij, R., Hochstein, L., Kosewski, L., Reynolds, J., & Rosenthal, C. (2016). Chaos engineering. *IEEE Software*, 33(3), 35–41. <https://doi.org/10.1109/MS.2016.60>

Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (2016). *Site reliability engineering: How Google runs production systems*. O'Reilly Media.

Beyer, B., Murphy, N. R., Rensin, D. K., Kawahara, K., & Thorne, S. (2018). *The site reliability workbook: Practical ways to implement SRE*. O'Reilly Media.

LitmusChaos. (s. f.). *Cloud native chaos engineering*. Recuperado el 29 de agosto de 2026, de <https://litmuschaos.io/>

Netflix. (s. f.). *Chaos Monkey*. Recuperado el 29 de agosto de 2026, de <https://netflix.github.io/chaosmonkey/>

Principles of Chaos Engineering. (s. f.). *Principles of chaos engineering*. Recuperado el 29 de agosto de 2026, de <https://principlesofchaos.org/>

Shopify. (s. f.). *Toxiproxy: A TCP proxy to simulate network and system conditions*. Recuperado el 29 de agosto de 2026, de <https://github.com/Shopify/toxiproxy>